RF*diffusion*

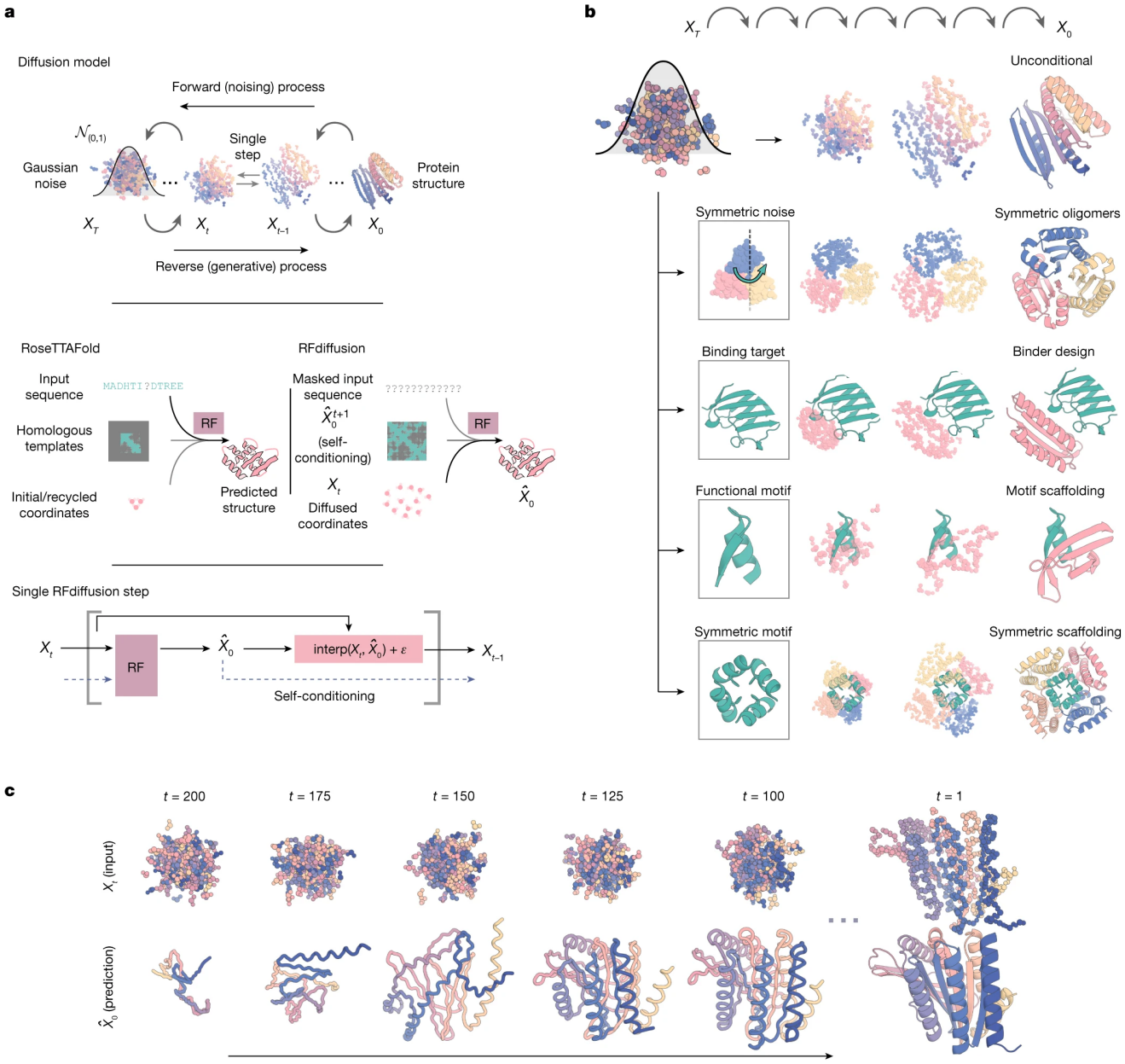


Image: RFdiffusion Paper

简介

一个开源的骨架生成模型，可以基于模板或者无模板生成不包含序列信息的单体蛋白、结合蛋白、同源多聚体蛋白的骨架等 the RFdiffusion paper.

RFdiffusion可以做什么

- 从头生成一个单体蛋白
- 基于一段基序生成骨架
- 从头生成具有对称性的多聚体蛋白 (cyclic, dihedral, tetrahedral symmetries)
- 基于一段基序生成具有对称性的多聚体蛋白
- 设计结合蛋白 (基于模板或从头生成)
- 骨架扰动 ("partial diffusion"，以已有骨架为起点进行更多的构象采样)

使用以及安装

Sergey Ovchinnikov 提供了一个便于使用的Google Colab Notebook 可以无需安装直接使用。

本地安装可以参考以下步骤：

首先克隆RFdiffusion的内容到本地：

```
git clone https://github.com/RosettaCommons/RFdiffusion.git
```

你需要将以下的模型参数文件下载到RFdiffusion目录下.

```
cd RFdiffusion
mkdir models && cd models
wget
http://files.ipd.uw.edu/pub/RFdiffusion/6f5902ac237024bdd0c176cb93063dc4/Base_ckpt
.pt
wget
http://files.ipd.uw.edu/pub/RFdiffusion/e29311f6f1bf1af907f9ef9f44b8328b/Complex_b
ase_ckpt.pt
wget
http://files.ipd.uw.edu/pub/RFdiffusion/60f09a193fb5e5ccdc4980417708dbab/Complex_F
old_base_ckpt.pt
wget
http://files.ipd.uw.edu/pub/RFdiffusion/74f51cfb8b440f50d70878e05361d8f0/InpaintSe
q_ckpt.pt
wget
http://files.ipd.uw.edu/pub/RFdiffusion/76d00716416567174cdb7ca96e208296/InpaintSe
q_Fold_ckpt.pt
wget
http://files.ipd.uw.edu/pub/RFdiffusion/5532d2e1f3a4738decd58b19d633b3c3/ActiveSit
e_ckpt.pt
wget
http://files.ipd.uw.edu/pub/RFdiffusion/12fc204edeae5b57713c5ad7dcb97d39/Base_epoc
h8_ckpt.pt
```

Conda 虚拟环境的搭建

使用 Anaconda 或者 Miniconda 建立python虚拟环境.

以Anaconda为例：

```
conda env create -f env/SE3nv.yml

conda activate SE3nv
cd env/SE3Transformer
pip install --no-cache-dir -r requirements.txt
```

```
python setup.py install
cd ../.. # 切换到安装目录
pip install -e . # 安装模块
```

当你想要使用RFdiffusion的时候需要激活环境（尽量使用 CUDA 11.1版本）：

```
conda activate SE3nv
```

使用之前需要设计提供调用脚本的环境变量:

```
source ~/.rfdiffusionrc
```

这个环境变量文件提供了运行脚本的路径，示例如下：

```
export PATH=$PATH:""
```

使用示例

如何运行diffusion脚本

脚本的路径位于 `scripts/run_inference.py`。有许多不同的方式可以控制这个脚本的参数，比如通过 Hydra configs 可以非常细致的定义推理过程中的参数，这个默认的控制文件位于 `config/inference/base.yaml` 。参数文件调用位于`run_inference.py`脚本的第38行：

```
@hydra.main(version_base=None, config_path="../config/inference",
config_name="base")
```

---

1. 基础使用 - 从头生成一个单体蛋白

以一个100个氨基酸的蛋白质生成为例，我们需要考虑以下内容：

1. 生成的蛋白序列长度（100）
2. 生成骨架的数量
3. 输出结果的路径

```
run_inference.py 'contigmap.contigs=[150-150]'
inference.output_prefix=test_outputs/test inference.num_designs=10
```

详细讲解内容

首先，什么是 `contigmap.contigs`？ Hydra 配置文件决定了推理脚本如何运行。为了保持结构清晰，配置文件被划分成不同的子配置，其中一个子配置是 `contigmap`，它与定义构建蛋白的连续字 符串有关。

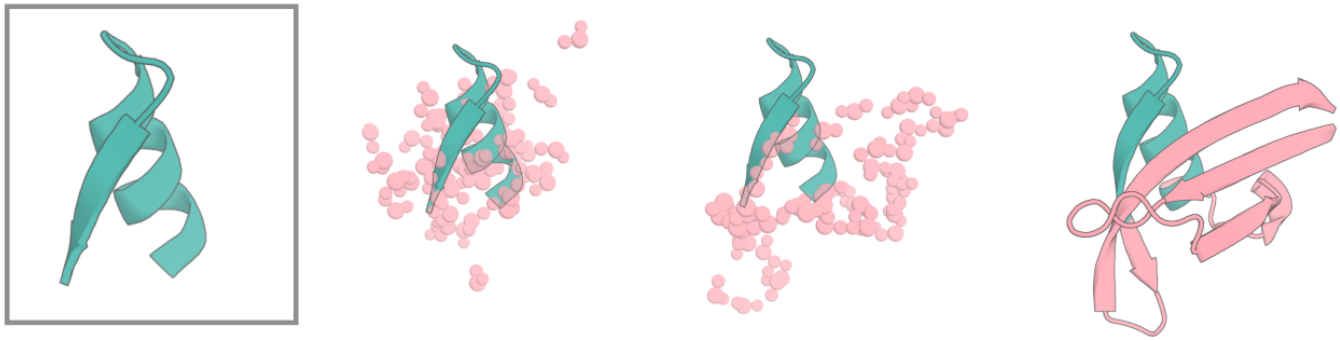
```
contigmap:
  contigs: null
  inpaint_seq: null
  provide_seq: null
  length: null
```

配置文件中的任何内容都可以通过命令行手动覆盖。例如：

```
 contigmap.inpaint_seq=[A163-168/A170-171/A179]
```

---

2. 基于一段基序生成骨架



我们需要指定如何将特定的蛋白质输入（来自 `.pdb` 文件的一个或多个片段）进行骨架化，并定义如何连接这些片段以及新骨架蛋白中的残基数量。此外，我们希望能够对连接蛋白的长度进行采样，因为通常我们事先并不知道需要多少残基才能最好地支架化一个基序。这些输入的指定由 **contigs** 完成，并由 Hydra 配置文件中的 **contigmap** 配置控制。

简单示例

- 在这里，我们运行一个motif-scaffolding基准测试的示例。
- 具体来说，我们对RSV-F蛋白的片段进行scaffolding。
- 其逻辑是，只有部分氨基酸对motif的功能重要，而其他可以重新设计。
- 这可以促进更好的包装，因为RFdiffusion不会被迫与motif背面不重要的特定残基进行对接。
- 我们指定输出路径和输入的PDB文件（RSV-F蛋白）。
- 我们通过contig输入指定想要构建的蛋白质：
  - 10到40个残基（随机采样）
  - 输入中A链的残基163-181（包括在内）
  - 10到40个残基（随机采样）
- 生成10种设计。
- 指定在输入的A链中，残基163-168、170-171和179需要被掩蔽。

```
run_inference.py
inference.output_prefix=example_outputs/design_motifscaffolding_inpaintseq
inference.input_pdb=input_pdbs/5TPN.pdb 'contigmap.contigs=[10-40/A163-181/10-
40/0]' inference.num_designs=10 'contigmap.inpaint_seq=[A163-168/A170-171/A179]'
```

简要说明

1. **带字母前缀的项**：表示这是一个基序，前缀字母对应输入 `.pdb` 文件中的链标。例如，`A10-25` 表示输入 pdb 中链 `A` 的残基 10 到 25（即，`('A',10), ('A',11)... ('A',25)`）。
2. **不带字母前缀的项**：表示**需要构建的蛋白**。这可以用一个长度范围表示。每次运行 RFdiffusion 推理时，这些长度范围会随机采样。
3. **链断裂**：用 `/0` 指定。

`inpaint_seq` 标签

- 比如在融合两个蛋白质时，原本位于蛋白质表面的残基（通常是极性的）可能需要被重新打包到蛋白质的"核心"中，因此需要转变为疏水残基。
- 我们可以屏蔽其序列特性，允许 RFdiffusion 隐式推断其序列，从而更好地对其进行折叠。

屏蔽残基序列时，可以使用以下语法：

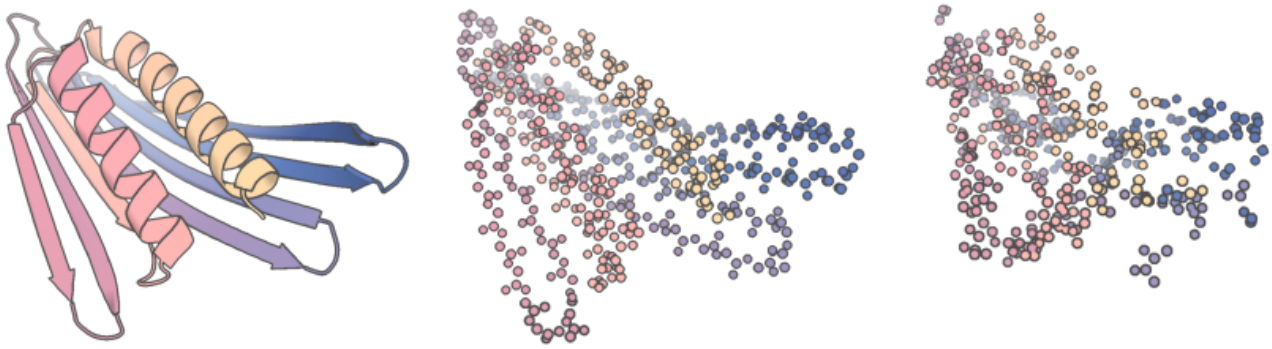
```
'contigmap.inpaint_seq=[A163-168/A170-171/A179]'
```

这里我们屏蔽了残基 A163-168, A170-171以及 A179的序列特性。

---

3. Partial diffusion

有时我们可以使用扩散模型对一个结构进行部分的加噪和去噪,从而基于这个起始结构获得更加多样的骨架。可以参考Vazquez-Torres et al., BioRxiv 2022的工作。 我们可以通过 `diffuser.partial_T` 标签来控制加噪的步数。



简单示例

- 这里，我们从一个初始结构开始，部分地加入噪声并去噪声，以生成多样性的骨架。
- 首先，我们需要指定输出路径以及输入的 79 个氨基酸的 PDB 文件路径（2KL8）。
- 接着，我们需要指定与需要添加噪声部分相对应的片段（contig）。
- 在本例中，我们希望对整个结构添加噪声，因此指定为 [79-79]，即对所有 79 个残基进行噪声处理。
- 我们也可以仅对后 59 个残基添加噪声，这时需要指定为 [A1-19/59]。
- 无论选择哪种方式，片段的总长度必须与输入 PDB 文件的长度相加一致。
- 最后，我们生成 10 个设计，并进行 10 步加噪和去噪声操作（相当于全路径的 20%,10/50）。
- 更多的加噪意味着更大的多样性。

```
run_inference.py inference.output_prefix=example_outputs/design_partialdiffusion
inference.input_pdb=input_pdbs/2KL8.pdb 'contigmap.contigs=[79-79]'
inference.num_designs=10 diffuser.partial_T=10
```

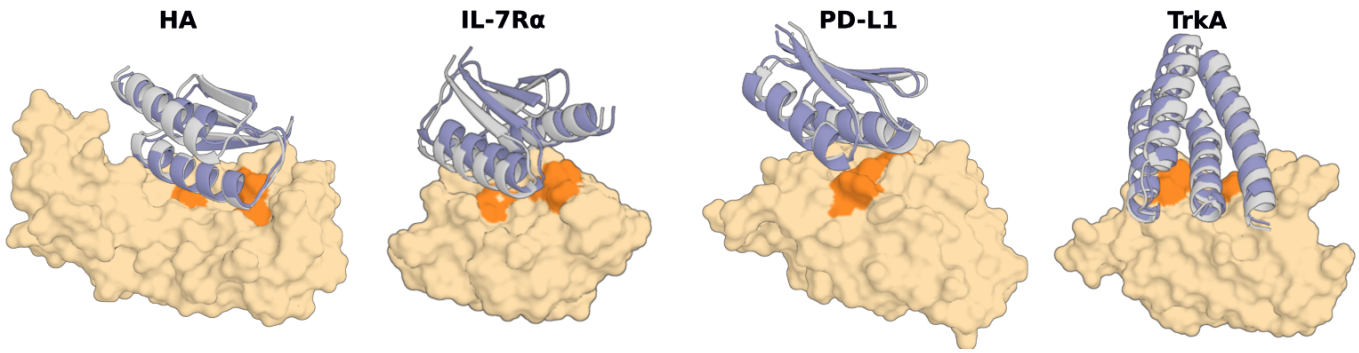
RFdiffusion方法默认的扩散步数是50步，如果`diffuser.partial_T`设置为50意味着完全重新扩散了一个蛋白骨架。 如果想要尝试更大的步数如`80`，那么需要把`diffuser.T`设置的更大例如`200`。这其实和`20/50`是相同的。

Partial diffusion方法只会产生和输入结构一样长的结果，但是也可以使用contigs控制输入。例如输入一个binder：target复合物，我们对binder（Chain A， 100个氨基酸）进行多样化，我们可以使用下面的方式：

```
'contigmap.contigs=[100-100/0 B1-150]' diffuser.partial_T=20
```

---

4. Binder Design



如果chain B是你的靶点，接下来可以这样做,

```
run_inference.py 'contigmap.contigs=[B1-100/0 100-100]'
inference.output_prefix=test_outputs/binder_test inference.num_designs=10,
```

这样会靶向chain B 1-100的残基，生成10个长度为100个氨基酸的binder。

随机的生成结合蛋白往往不是我们所期望的结果。

我们通过向模型提供我们所谓的"热点残基"来解决这个问题。指定hotspot：`'ppi.hotspot_res=[A30,A33,A34]'`

一个用RFdiffusion做binder design的例子

- 不指定binder的拓扑结构（不基于模板）。
- 指定靶蛋白（insulin receptor）的路径。
- 描述输入的contig：
  - 指定使用的靶蛋白，链A的1-150个氨基酸。
  - 设置一个chain break，因为binder不可以融合在靶蛋白中。
  - 生成一个70-100个氨基酸的binder。
- 设置热点氨基酸为靶蛋白的“59，83，91”位氨基酸。
- 生成10个binder

```
run_inference.py inference.output_prefix=example_outputs/design_ppi
inference.input_pdb=input_pdbs/insulin_target.pdb 'contigmap.contigs=[A1-150/0 70-
100]' 'ppi.hotspot_res=[A59,A83,A91]' inference.num_designs=10
denoiser.noise_scale_ca=0 denoiser.noise_scale_frame=0
```

---

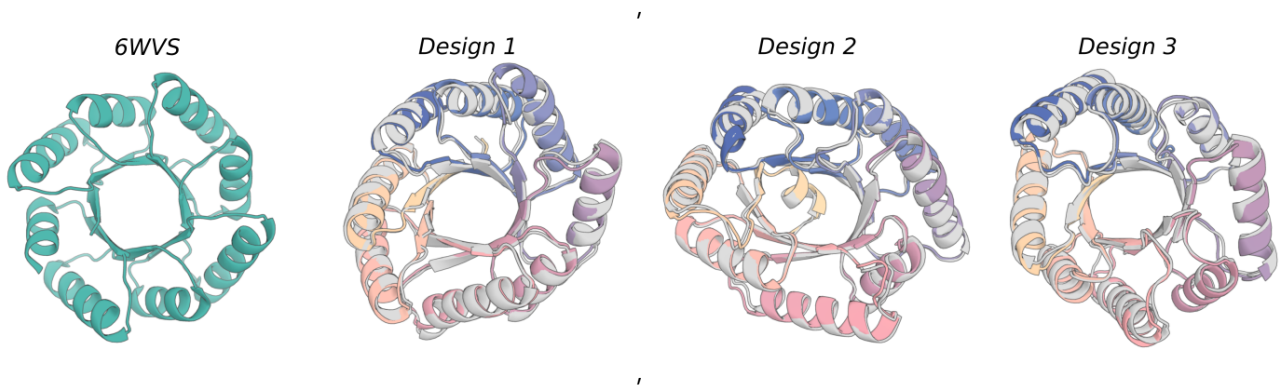
5. Practical Considerations for Binder Design

common pitfalls

- Selecting a Target Site
  - 有至少3个疏水残基能与binder互作
  - 避免带电荷的极性位点，带有聚糖分子或者聚糖分子附近的位点，非结构化的loop
- Truncating your Target Protein
  - RFdiffusion运行时间(N^2)N是氨基酸的数量
  - 保留二级结构并引入一些chain breaks
  - 在靶点上想要结合界面留大约10A
- Picking Hotspots
  - 疏水残基
  - 模型会获取0-20%的hotspots的信息，其余的会被mask，建议选择3-6个
- Binder Design Scale
  - 每一个target生成了大约10,000 RFdiffusion binder backbones
  - 对于一些target而言，可能生成1000个RFdiffusion backbones就足够了，你需要的是得到足够的design使它们 pAE_interaction < 10
- Sequence Design for Binders
  - RFdiffusion生成的binder序列都是glycine
  - ProteinMPNN-FastRelax设计序列
- Binder Design Filtering

    - 评估设计的binder是否可能有功能，用AF2预测计算pae

- Fold Conditioning
    - 让binder或者monomer限制在某个特定的拓扑结构：提供二级结构和相邻信息



从PDB结构生成input

```
./make_secstruc_adj.py --input_pdb ./2KL8.pdb --out_dir /my/dir/for/adj_secstruct
```

or

```
./make_secstruc_adj.py --pdb_dir ./pdbs/ --out_dir /my/dir/for/adj_secstruct
```

可以处理一个PDB或者一个含有多个PDB的文件夹，然后输出二级结构和相邻信息的文件,

```
./scripts/run_inference.py
inference.output_prefix=./scaffold_conditioned_test/test
scaffoldguided.scaffoldguided=True scaffoldguided.target_pdb=False
scaffoldguided.scaffold_dir=./examples/ppi_scaffolds_subset
```

另外，对于蛋白-蛋白相互作用：

- 需要提供目的蛋白，以及它二级结构和相邻信息的文件：

```
scaffoldguided.target_pdb=True
scaffoldguided.target_path=input_pdbs/insulin_target.pdb
inference.output_prefix=insulin_binder/jordi_ss_insulin_noise0_job0
'ppi.hotspot_res=[A59,A83,A91]'
scaffoldguided.target_ss=target_folds/insulin_target_ss.pt
scaffoldguided.target_adj=target_folds/insulin_target_adj.pt
```

- 加上以下flag：

```
scaffoldguided.mask_loops=False
```

在训练过程中会mask一些二级结构和相邻信息，这样可以轻易的加入一些额外的氨基酸，不用管它精确的二级结构，比如，你想生成一个长的3个helix的bundle，可以mask loop然后对这个loop加上比如20额外的'mask' tokens

```
scaffoldguided.mask_loops=True scaffoldguided.sampled_insertion=15
scaffoldguided.sampled_N=5 scaffoldguided.sampled_C=5
```

将对最多15个残基进行采样以插入3HB输入的循环中，并在N和C末端最多额外采样5个残基。 在没有大量pdb来创建相邻信息是非常有用。例如，使用这种策略，我们可以从单个起始pdb生成加长的TIM barrel。最后,如果你有一个大目录的相邻信息/二级结构文件,但不想使用所有这些文件,你可以制作一个你想要使用的.txt文件：

```
scaffoldguided.scaffold_list=path/to/list
```

对于蛋白-蛋白相互作用，减少noise可以适当improve design：

```
denoiser.noise_scale_ca=0.5 denoiser.noise_scale_frame=0.5
```

另一个例子：`./examples/design_ppi_scaffolded.sh`，RFdiffusion设计flexible peptides的binder，提供peptide的pdb结构但mask相应的3D结构：

```
inference.input_pdb=input_pdbs/tau_peptide.pdb 'contigmap.contigs=[70-100/0 B165-
178]' 'contigmap.inpaint_str=[B165-178]'
```

指定这段结构为beta strand：

```
scaffoldguided.scaffoldguided=True 'contigmap.inpaint_str_strand=[B165-178]'
```

---

6. Using Auxiliary Potentials

在denoising中用辅助potential来guide diffusion，比如让设计的蛋白达到想要的shape，well-packed protein。

- 使用一连串string来指定`potentials.guiding_potentials`，有多个：`arg_name1:arg_value1,arg_name2:arg_value2,...,arg_nameN:arg_valueN`
- 用不同的decays (`constant`,`linear`,`quadratic`,`cubic`)， `potentials.guide_decay`
- 一个例子：

```
potentials.guiding_potentials=
[\\\type:olig_contacts,weight_intra:1,weight_inter:0.1\\\]
potentials.olig_intra_all=True potentials.olig_inter_all=True
potentials.guide_scale=2 potentials.guide_decay='quadratic'
```

一开始建议不加potential， 在过程中逐步增加合适的potential。